

AnnoGram: An Annotative Grammar of Graphics Extension

Md Dilshadur Rahman , Md Rahat-uz- Zaman , Andrew McNutt , and Paul Rosen 

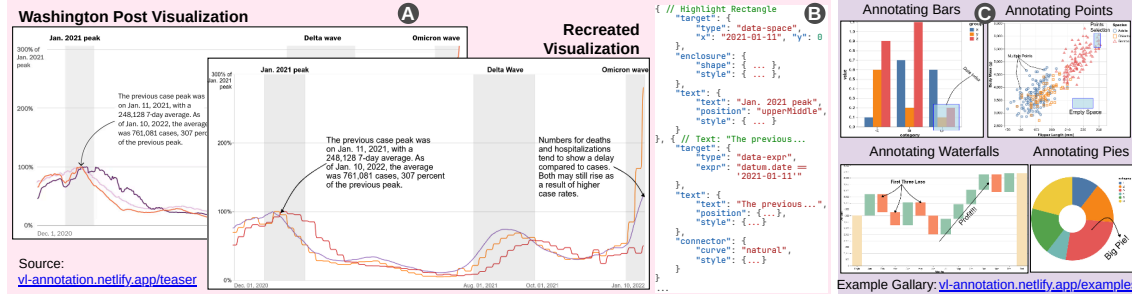


Figure 1: (A) An annotated Washington Post visualization [36] recreated using a proof-of-concept implementation of our AnnoGram extension to the Grammar of Graphics. (B) is shown alongside an excerpt of the specification. (C) the implementation supports diverse annotation types (text, connector, enclosure, indicator, etc.) across various chart types.

ABSTRACT

Annotations are central to effective data communication, yet most visualization tools treat them as secondary constructs—manually defined, difficult to reuse, and loosely coupled to the underlying visualization grammar. We propose a declarative extension to Wilkinson’s Grammar of Graphics that reifies annotations as first-class design elements, enabling structured specification of annotation targets, types, and positioning strategies. To demonstrate the utility of our approach, we develop a prototype extension called Vega-Lite Annotation. Through comparison with eight existing tools, we show that our approach enhances expressiveness, reduces authoring effort, and enables portable, semantically integrated annotation workflows.

Keywords: annotation, visualization grammar

1 INTRODUCTION

Annotations are textual and graphical elements embedded in visualizations to draw attention, highlight salient features, provide context, and guide interpretation [22, 27, 30]. They improve memorability, comprehension, and recall [1, 3, 4, 7], and help externalize insights during analysis [15, 19, 34]. Annotations are employed in visualizations across numerous domains, including scientific research, business analytics, healthcare, education, journalism, and public policy. Despite their ubiquity, current programmatic tools do not treat annotations as first-class elements, making annotations difficult to reuse across different designs and datasets and impeding their computational analysis (such as for recommendation).

Most visualization platforms treat annotations as secondary overlays, offering only ad hoc mechanisms loosely connected to underlying data and encodings. For example, Vega-Lite [32] lacks a dedicated annotation abstraction; users must instead define separate data sources and specify annotation marks using basic graphical primitives such as points, lines, and text. This approach is verbose, error-prone, and complicates the maintenance and updating of annotations as the data or design evolves. Similarly, D3.js [5] affords low-level graphical control but requires users to imperatively construct, position, and manage annotations, tightly binding them to specific chart implementations and reducing reusability. Although tools such as d3-annotation offer reusable task-specific components, their non-declarative design limits flexibility and generalization across visualization types. The situation is similar for other programmatic tools. As a result, many practitioners export charts to tools such as

PowerPoint to add annotations manually [2, 23], severing links to data, removing interactivity, and limiting portability and automation.

To address these challenges, we propose AnnoGram, an extension to Wilkinson’s Grammar of Graphics [39] (GoG) that elevates annotation to a first-class design element—decomplexing it from GoG’s notion of guides and secondary datasets, and making it a sibling to scales and geometries. Our extension supports declarative specification of annotation types, targets, positioning strategies, and styles. This support reduces manual positioning, enables data-driven placement, and promotes reusability, allowing annotations to transfer across charts as needs evolve. As a demonstration, we construct a proof-of-concept implementation that extends Vega-Lite [32]. By extending this DSL, rather than re-implementing it, we can reuse the full Vega rendering pipeline while still making use of it as a GoG petri dish (echoing Zong, Pollock, et al. [42]). We evaluate the extension using a heuristic-based comparison with existing tools.

Through this work, we show that annotations can be first-class components of visualization grammars that enabling consistent, reusable workflows.

2 RELATED WORK

Our work extends prior work on tool support for visualization annotations as well as on Grammar of Graphics [39]. GoG provides an expressive conceptual framework for describing a wide array of statistical graphics and has been used to structure tools like ggplot2 [38] and Vega-Lite [32]. The central observation in this model is that specifying charts via their types (e.g., scatterplots or bar charts) complements different ideas (e.g., notions of scale and encoding) that, when reified, support expressive construction of a broad range of charts, both novel and familiar. Yet, this model is not perfect and omits important structures for some tasks. For instance, accessibility [21] or uncertainty [25], or as we consider, annotation. Notably, GoG can support each of these tasks in an unmodified form (cf. Wilkinson’s uncertainty chapter [39]); however, it encounters the so-called *Turing tarpit*, a situation in programming language design in which “everything is possible but nothing of interest is easy” [24]. This situation is particularly evident for annotation, where current GoG-based tools require verbose and indirect encodings. We develop an extension that privileges a specific task of interest (annotation in our case) that, based on usage [26], is believed to be important.

Recent efforts have extended Vega-Lite’s declarative grammar to support underrepresented design concerns—an approach our work follows for annotations. For instance, Cicero [16] is a responsive design grammar that modifies Vega-Lite to enable layout adaptation across devices. Animated Vega-Lite [42] extends the Vega-Lite’s compiler with a temporal encoding channel to support animation, treating time as a first-class visual variable. Tactile Vega-Lite [8]

```

Root
{
  target := (id | FixedPos | ChartPart | DataPoint | Axis | None)[]
  text   := Annotation(TextAnn)
  enclosure := Annotation(EnclosureAnn)
  connector := Annotation(ConnectorAnn)
  indicator := Annotation(IndicatorAnn)
  ...
}

Annotation Targets
DataPoint := Expression | index[]
ChartPart := title | legend | subtitle | caption | ...
Axis      := {
  axis := x | y, parts := label | tick | grid | tick-label
  range := Coord[] | Expression
}

Annotation Types
Annotation(X) := (id, Style, X), Style = ...
TextAnn      := (string, Position | None)
EnclosureAnn := (Rect | Ellipse | SVGShapePath | ..., Position | None)
ConnectorAnn := (Markers | None, SVGShapePath, linear | catmull-rom | ...)
IndicatorAnn  := (line | area | arrow | ..., Expression)

Placement
Position := FixedPos | Anchor1D | Anchor2D
Anchor1D := auto | start | mid | end, Anchor2D := auto | upLeft | midRight | ...
FixedPos  := {type := data | pixel, x := Coord, y := Coord}
Coord     := string | number

```

Figure 2: AnnoGram introduces a top-level primitive to declaratively define annotation targets, types, and placement.

augments the grammar with tactile-specific abstractions—such as braille, textures, and spatial layout controls—to support blind and low-vision readers. DXR [35] uses an extension of Vega-Lite syntax to support authoring immersive 3D visualizations. We similarly extend the Vega-Lite grammar but focus on the underexplored dimension of annotation, reifying it as a core design element. In doing so, our work complements these prior extensions by addressing a different yet equally foundational aspect of visualization design.

Beyond grammar-based approaches, many tools support annotations [9, 12, 13, 40], albeit in limited ways [26]. For example, ChartAccent [31] maps data to marks using rules. Contextifier [14] automatically generates annotations from text for time series charts. Although these systems support annotations, they commonly treat them as external overlays detached from the underlying visualization model, lacking the modularity, portability, and integration commonly found in data-driven workflows. Our work enables annotations to be specified, reasoned about, and rendered alongside core visualization constructs. Some prior work has also explored structured annotations for reasoning [41] and narrative generation [6], treating annotations as integral to analysis or storytelling. Our work complements these by formalizing annotation structure for declarative specification and reuse.

3 EXTENSION DESIGN

Our primary contribution is AnnoGram (summarized in Fig. 2), an extension to GoG that elevates annotations to the same level as other primary GoG components, such as geometries or scales. Unlike other extensions, ours is complicated by the issue that annotations require information from across the specification-assembly-display pipeline (a concept underpinning the organization of GoG [39] itself). For instance, annotation placement can require knowledge of the pixel position of generated marks (such as to address overlap or collision) while also needing knowledge of the data (such as to select the maximum value in a scatterplot).

Given these complications, we choose to frame our extension in the manner of CSS via a *target-and-effect* style. Each annotation has a *target* and associated visual effects (each with its own positioning and styling elements). For instance, an annotation might target a specific bar in a bar chart, attach an arrow to it, and then a textual explainer might note that it is an outlier. This model aligns with a likely common authoring thought-process: first, an annotation author selects target elements or data points to annotate; next, they choose the type of annotation to apply (e.g., text, enclosure, or indicator); finally, they may optionally customize the annotation with styling, positioning, or connectors. We reify this process as collec-

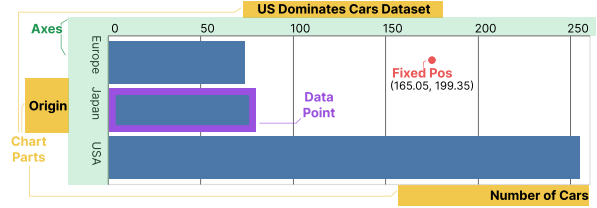


Figure 3: Supported annotation targets.

tions of annotations that select targets, categorizing annotation types, determining placement, and specifying annotation styles. This declarative design supports integration into GoG, but is more closely aligned with our proof-of-concept implementation and design choices made in Cicero [16].

We developed AnnoGram through an iterative process of analyzing common annotation practices [27] and encoding them as first-class constructs in a declarative grammar. Our goal was to align this grammar closely with the needs of implementation, ensuring that it is expressive enough for diverse annotation types yet tractable within for typical rendering pipelines. The remainder of this section describes the high-level elements of our design.

Target. An annotation must first define its *target*: the specific visualization element it applies to, of which there are several types (see Fig. 3). First are **DataPoints**, which enable annotations to highlight individual data points or subsets defined using direct indexing or expressions. For example, an annotation can highlight all values exceeding a temperature threshold, updating dynamically as the data changes. Beyond data points, annotations can target other key components of a chart. **Axis** targets allow annotations to be applied to the x- or y-axis, marking significant values, ranges, or structural elements such as tick marks, gridlines, and labels. For instance, an annotation might highlight a time range on the x-axis to provide historical context. **ChartPart** targets are situating elements such as titles, legends, and axis labels, supporting meta-level explanations such as clarifying what colors or shapes represent in a legend. In some cases, annotations are not tied to any specific chart element (i.e., *None*) and instead serve as standalone notes, positioned independently to offer supplementary context.

Annotations can also interact with each other to create structured, layered explanations. For example, a shaded region might group a set of chart elements, while a nearby explanatory label provides a description of the group’s significance. To formalize these relationships, we define two mechanisms: **Reference** and **Composite**. In a **Reference** relationship, one annotation explicitly links to another via its *id*—for example, a marker denoting an anomaly might reference a separate annotation that explains its cause. **Composite** annotations, in contrast, group multiple annotations together to describe repeated structures or patterns in the data—such as highlighting seasonal trends across several time intervals using coordinated shapes, markers, and labels. Both of these relationships reflect the concept of annotation ensembles [27–29], in which multiple annotations are intentionally combined to convey complex meanings that single annotations cannot express on their own.

Annotation Types. Once a target has been defined, the next step is to determine how it should be annotated. Should the annotation appear as a text label, a visual enclosure, or some other graphical element? We support a broad set of annotation types based on Rahman et al.’s [27] taxonomy. Our grammar accommodates all seven types described in their study, but we focus here on those used in our proof-of-concept implementation. **Text** annotations convey explanatory context for specific data points, chart regions, or encodings (e.g., text descriptions in Fig. 1A). They are highly customizable, with configurable properties such as font size, color, and alignment. **Enclosure** annotations draw attention to clusters or regions of interest using shapes such as rectangles, ellipses, brackets, or curly braces (e.g., gray highlighted area in Fig. 1A). These annotations support adjustable padding and stroke styles, offering flexibility in

visually presented grouped elements. **Indicator** annotations mark thresholds, trends, or events through visual markers such as lines, arrows, or highlights (e.g., the arrow showing upward trend of profit in the waterfall chart in Fig. 1C). Styling options include stroke color, width, and arrowheads. **Connector** annotations link visual elements (e.g., data points and explanations) using paths or lines (e.g., arrows connecting text descriptions with points of interest in Fig. 1A). They support various interpolation modes, including linear, curved, and stepwise with other styles.

Placement. The last key consideration is placement. While **target** implies constraints—annotations should remain visually associated with their referent—additional positioning strategies offer greater flexibility. We include a set of placement defaults that users can override, similar to how inline CSS can take precedence over external stylesheets (see Fig. 4.3). We include two primary positioning modes. **Fixed Positioning** places annotations in either Data-Space or Pixel-Space. The former anchors annotations relative to encoding scales, enabling reuse across chart types; the latter offers absolute control via screen coordinates. Fine-grained adjustments are supported via dx/dy offsets (not shown in Fig. 2). Alternatively, **Relative Positioning & Anchoring** aligns annotations dynamically with their targets. Anchors may apply in one or both dimensions, snapping to element edges or centers. Importantly, manual positioning is never required as annotations receive meaningful placement automatically based on target and layout context, but authors retain full control to specify or refine positions when desired. This flexibility increases the utility of automated strategies, ensuring authors can rely on default behaviors while selectively applying manual overrides where precision or customization is needed. This dynamic alignment ensures that annotations remain meaningfully positioned as chart dimensions or layouts shift.

4 PROOF-OF-CONCEPT IMPLEMENTATION

We developed a proof-of-concept demonstration of AnnoGram as a Vega-Lite extension to explore the utility of our design.

Architecture. Our proof-of-concept extends Vega-Lite’s compilation pipeline to integrate annotations into the specification and rendering process. Annotations require coordination between semantic targets (e.g., data points, axes, chart parts) and their placement in the rendered scene. The Vega-Lite pipeline, including normalization, scene graph generation, and rendering, is executed in parallel to extract styles and coordinates, which inform default styling and placement for coherent annotation design with minimal user input. As in Fig. 4, our library processes annotations in several stages: a **Parser**, a **Position Resolver**, and an **Assembler**, followed by rendering via a **Transpiler** and **Post-Adder**.

The **Parser** takes the input JSON specification, validates it, extracts annotations, and resolves syntax, scales, and default values from the normalized Vega-Lite and Vega specifications. The **Position Resolver** then analyzes the scene graph to determine optimal annotation placement. User-defined positions (e.g., **FixedPos**) are prioritized; otherwise, it heuristically selects available space to minimize occlusion (echoing Vega-Label [37]’s placement strategy). For example, in the scatterplot shown in Fig. 4.2, it identifies available regions (green) and places annotations in unoccupied areas (yellow) to avoid overlap while accounting for anchoring and offset logic. The **Assembler** links related marks as needed (e.g., drawing connectors between labels and enclosures) and iteratively resolves circular or nested references. This coordination with the **Position Resolver** ensures spatial and semantic coherence across annotation elements.

Next, the **Transpiler** converts annotations into Vega mark encodings using graphical primitives (e.g., paths, shapes, and text). These are inserted into the Vega specification’s marks and data arrays alongside standard chart elements. Finally, the **Post-Adder** handles annotations requiring advanced rendering, such as non-data-space placement or complex shapes. Elements such as ellipse-shaped enclosures are directly injected into the Vega scene graph with absolute pixel coordinates.

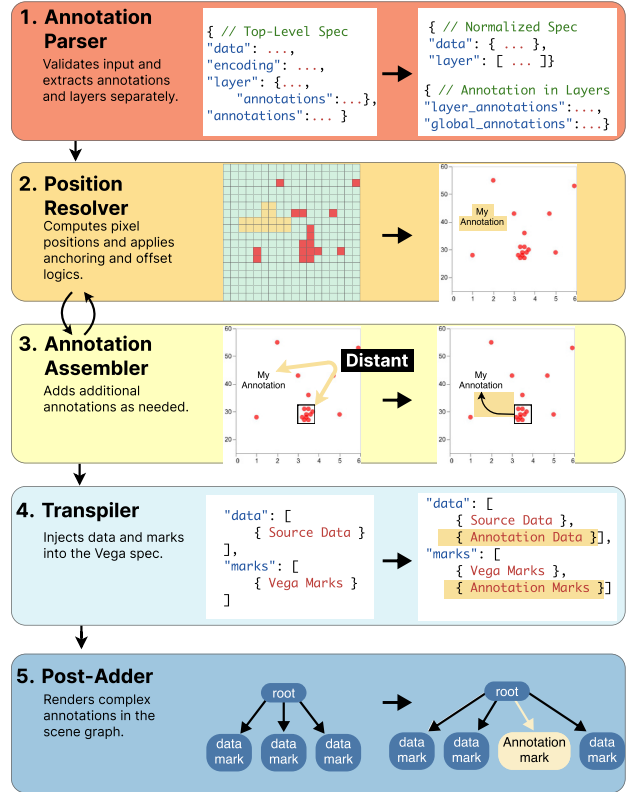


Figure 4: Workflow overview of AnnoGram implementation.

Data Dependent Transferable Annotations. A key contribution of our annotation grammar is support for annotation portability—annotations that remain valid and well-positioned as data or chart types change. This portability is achieved through a combination of user-defined and automatic positioning strategies that decouple annotations from specific mark or layout configurations.

For user-defined positioning, the **Annotation Parser** handles both pixel-space and data-space targets. Pixel-space targets are resolved via the scene graph for exact placement, whereas data-space targets use encoding scales extracted from the Vega specification. The **Position Resolver** computes annotation positions based on target type, specified positions, 10 mark types, and chart layout. The **Position Resolver** and **Annotation Assembler** iteratively assign positions by identifying unoccupied regions and minimizing occlusion via a backtracking algorithm. During the positioning, relative adjustments using offsets and anchor values (**Anchor1D**, **Anchor2D**) are also applied on the data between **Position Resolver** and **Assembler**. This differs from Vega-label’s [17, 37] approach, which is limited to repositioning only text annotations by adjusting text mark placements to avoid collisions. Whereas our approach supports multiple annotation types (text, enclosure, connector) with flexible, data- and pixel-based positioning across chart types.

These design choices enable portability by resolving annotation placement at runtime based on target semantics, chart encodings, and available space, making annotations robust to changes in chart structure, reducing the need for manual adjustments during design. For example, a text annotation targeting a data point in a scatterplot can be reused in a bar chart of the same data without any changes. In Fig. 5, a single annotation specification results in three different, context-appropriate placements, depending on the target and chart configuration.

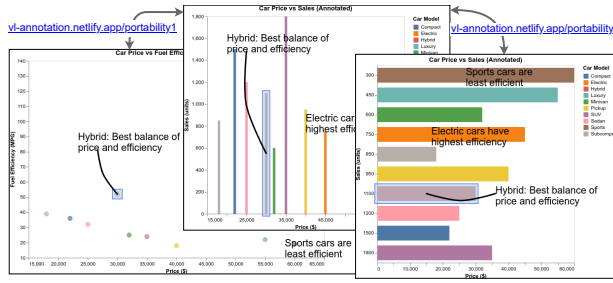


Figure 5: Same annotation specification applied to three different chart types with different fields of the same data. AnnoGram dynamically detects the target mark types and adds annotations.

5 EVALUATION

We conducted a heuristic comparative evaluation of our proof-of-concept implementation against eight existing tools. We do so by considering the differences between annotated bar, line, and scatterplots produced using each tool. Evaluation focused on annotation-specific and total lines of code (LOC) and four qualitative dimensions: **1st-class annotation support**, which assesses whether annotations are structured components within the visualization model rather than external overlays; **intuitiveness**, which measures how naturally the annotation syntax reflects the data and chart structure, and how easily common annotation tasks can be expressed; **error proneness** [10, 11], which captures the likelihood of mistakes during annotation authoring or editing, such as misalignment or broken references; and **portability**, which examines the ability of annotations to adapt across different datasets or chart types without manual rework. Table 1 summarizes the findings; examples and materials are available at github.com/shape-vis/codelines-comparison.

Across the evaluated tools, the degree of semantic integration was the primary factor shaping authoring complexity, intuitiveness, error susceptibility, and portability. General-purpose charting tools such as D3, Vega, and Vega-Lite lack built-in annotation constructs, requiring manual positioning, anchoring, and updates, resulting in high annotation LOC, cognitive load, and user error, with “Hard” ratings for intuitiveness and portability, and “High” error proneness. Whereas d3-annotate, ggplot2, and ggplot2-annotate introduce annotation primitives that reduce LOC and initial authoring effort, but weak semantic coupling still requires manual adjustments during updates or context changes. As reflected in their “Medium” ratings for intuitiveness and error proneness, these libraries partially improve authoring efficiency but do not fully address maintenance or dynamic adaptation challenges. HighCharts simplifies initial authoring but retains similar limitations, requiring user-driven updates for structural changes. Visual editors such as PowerPoint or Figma achieve “Easy” ratings for intuitiveness via direct manipulation, but disconnect annotations from data and chart structure, making them static and poorly portable across datasets or designs.

In contrast, AnnoGram embeds annotations as structured, semantically aware visualization components. Annotations are declaratively attached to data points, axes, and chart elements, enabling automated placement, anchoring, and runtime adaptation. This design substantially reduces annotation LOC compared to low-level libraries, minimizes manual corrections during data or layout changes, and results in consistently “Easy” ratings across all evaluated dimensions. Updates to the visualization automatically propagate to annotations, lowering error-proneness and enhancing portability without requiring reauthoring. Overall, the evaluation highlights a consistent pattern: tools without semantic annotation integration impose high authoring and maintenance overhead and are prone to errors during evolution. Systems that partially address authoring burden without semantic binding improve initial usability but struggle with long-term robustness. Our AnnoGram implementation demonstrates that embedding structured annotation semantics within the visualization grammar substantially improves authoring efficiency, reduces error rates, and supports scalable, maintainable annotation work-

Table 1: We implemented a trio of example charts with annotations across eight other tools. We give lines of code required for *just* the annotation (Ann.), as well as coding of several qualitative properties.

Programmability	Tool	Bar Ann. LOC	Bar LOC	Line Ann. LOC	Line LOC	Scatter Ann. LOC	Scatter LOC	1st-class Support	Intuitiveness	Error Proneness	Portability
High	d3 [5]	151	81	175	76	170	104	×	Hard	High	Hard
	d3-annotate [20]	161	81	174	77	177	99	✓	Med.	Med.	Med.
	ggplot2 [38]	120	22	128	43	208	101	✓	Med.	High	Med.
	ggplot2-annotate [38]	73	20	117	42	178	101	✓	Med.	High	Med.
Low	HighCharts [13]	82	32	231	82	169	79	✓	Med.	Low	Hard
	Vega [33]	515	161	491	219	387	186	×	Hard	High	Hard
	Vega-Lite [32]	259	23	253	29	177	34	×	Hard	High	Hard
	VL Annotation	101	19	95	25	79	31	✓	Easy	Low	Easy
–	Visual Editors (PowerPoint, Figma, etc.)	–	–	–	–	–	–	–	Easy	Low	Hard

flows across different datasets and visualization types. Additional consideration of the fine-grain differences between these notations is worthwhile future work, and may be aided by Kruchten et al.’s multi-notation gallery approach [18].

6 DISCUSSION

We present AnnoGram, an extension to GoG that treats annotations as first-class elements alongside encodings, marks, and layout, demonstrated via a proof-of-concept implementation as a Vega-Lite extension. Its declarative design enables runtime positioning and rendering, avoiding manual coordination. By referencing semantic targets and using scale-aware positioning, annotations remain portable across chart types and layouts, improving reusability, reducing effort, and minimizing errors compared to systems lacking integrated support (see Table 1).

Our proof-of-concept implementation realizes a subset of AnnoGram’s annotation capabilities. To ensure compatibility and ease of deployment, it is implemented as an external wrapper around Vega-Lite rather than modifying its compiler directly. This architectural choice simplifies integration but limits access to internal optimization and tooling infrastructure. In its current form, the library restricts each target to a single annotation per type, which reduces flexibility in scenarios that demand layered or redundant annotations. It also currently does not support compositions such as facet, layer, or concat, though these are planned for future versions. While some basic aggregations work in the current implementation, complex aggregations involving multiple mark types are not fully tested and may cause failures. Additionally, it supports only the most common annotation types observed in prior works [27], excluding less frequent or interactive forms. These decisions reduce complexity but constrain extensibility, especially for use cases needing richer semantics or native tooling support.

Although grounded in prior analysis [27], AnnoGram reflects one modeling approach that emphasizes simplicity and declarative clarity. Its current flat structure streamlines authoring and parsing, but limits expressiveness for nested or highly coordinated ensembles. Furthermore, our evaluation, based on a small set of expert-authored examples, also limits generalizability. Future work should explore alternative structures, broader usage contexts, and integration with recommendation systems and authoring tools. AnnoGram’s declarative structure opens promising directions for integration with annotation recommendation systems and tooling support. Its alignment with GoG concepts also suggests potential extensions to other under-supported constructs, such as explanatory annotations, visual embellishments, or accessibility layers. Embedding AnnoGram into grammar compilers could further enable native optimization, richer design semantics, and unified authoring environments.

ACKNOWLEDGMENTS

The authors wish to thank our anonymous reviewers. This work was supported in part by NSF award 2402719.

REFERENCES

- [1] S. Bateman, R. L. Mandryk, C. Gutwin, A. Genest, D. McDine, and C. Brooks. Useful junk? the effects of visual embellishment on comprehension and memorability of charts. In *SIGCHI Conference on Human Factors in Computing Systems*, pp. 2573–2582, 2010. doi: 10.1145/1753326.1753711 1
- [2] A. Bigelow, S. Drucker, D. Fisher, and M. Meyer. Iterating between tools to create and edit visualizations. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):481–490, 2016. doi: 10.1109/TVCG.2016.2598609 1
- [3] M. Borkin, A. A. Vo, Z. Bylinskii, P. Isola, S. Sunkavalli, A. Oliva, and H. Pfister. What makes a visualization memorable? *IEEE Transactions on Visualization and Computer Graphics*, 19(12):2306–2315, 2013. doi: 10.1109/TVCG.2013.234 1
- [4] M. A. Borkin, Z. Bylinskii, N. W. Kim, C. M. Bainbridge, C. S. Yeh, D. Borkin, H. Pfister, and A. Oliva. Beyond memorability: Visualization recognition and recall. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):519–528, 2015. doi: 10.1109/TVCG.2015.2467732 1
- [5] M. Bostock, V. Ogievetsky, and J. Heer. D³ data-driven documents. *IEEE Transactions on Visualization and Computer Graphics*, 17(12):2301–2309, 2011. doi: 10.1109/TVCG.2011.185 1, 4
- [6] C. Bryan, K.-L. Ma, and J. Woodring. Temporal summary images: An approach to narrative visualization via interactive annotation generation and placement. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):511–520, 2016. doi: 10.1109/TVCG.2016.2598876 2
- [7] A. Cedilnik and P. Rheingans. Procedural annotation of uncertain information. *IEEE Transactions on Visualization and Computer Graphics*, pp. 77–84, 2000. doi: 10.1109/VISUAL.2000.885679 1
- [8] M. K. Chen, I. Pedraza Pineros, A. Satyanarayan, and J. Zong. Tactile vega-lite: Rapidly prototyping tactile charts with smart defaults. In *SIGCHI Conference on Human Factors in Computing Systems*, 2025. doi: 10.1145/3706598.3714132 1
- [9] T. Eingartner, J. Drucker, and M. Dörk. Inflecting data visualizations for web-based scrollytelling. 2025. 2
- [10] T. R. G. Green. Instructions and descriptions: some cognitive aspects of programming and similar activities. In *Advanced Visual Interfaces*, 8 pages, p. 21–28. Association for Computing Machinery, New York, NY, USA, 2000. doi: 10.1145/345513.345233 4
- [11] T. R. G. Green and M. Petre. Usability analysis of visual programming environments: a ‘cognitive dimensions’ framework. *Journal of Visual Languages & Computing*, 7(2):131–174, 1996. doi: 10.1006/jvlc.1996.0009 4
- [12] J. Heer, F. Viégas, and M. Wattenberg. Voyagers and voyeurs: supporting asynchronous collaborative information visualization. In *SIGCHI Conference on Human Factors in Computing Systems*, pp. 1029–1038, 2007. doi: 10.1145/1240624.1240781 2
- [13] Highsoft AS. Highcharts documentation. Online, 2025. Last accessed: January 27, 2025. 2, 4
- [14] J. Hullman, N. Diakopoulos, and E. Adar. Contextifier: automatic generation of annotated stock visualizations. In *SIGCHI Conference on Human Factors in Computing Systems*, pp. 2707–2716, 2013. doi: 10.1145/2470654.2481374 2
- [15] Y.-a. Kang and J. Stasko. Characterizing the intelligence analysis process through a longitudinal field study: Implications for visual analytics. *Information Visualization*, 13(2):134–158, 2014. doi: 10.1177/1473871612468877 1
- [16] H. Kim, R. Rossi, F. Du, E. Koh, S. Guo, J. Hullman, and J. Hoffswell. Cicero: A declarative grammar for responsive visualization. In *SIGCHI Conference on Human Factors in Computing Systems*, pp. 1–15, 2022. doi: 10.1145/3491102.3517455 1, 2
- [17] C. Kittivorawong. Legible label layout for data visualization, algorithm and integration into vega-lite, 2024. 3
- [18] N. Kruchten, A. M. McNutt, and M. J. McGuffin. Metrics-based evaluation and comparison of visualization notations. *IEEE Transactions on Visualization and Computer Graphics*, 30(1):425–435, 2023. doi: 10.1109/TVCG.2023.3326907 4
- [19] H. Lin, D. Akbaba, M. Meyer, and A. Lex. Data hunches: Incorporating personal knowledge into visualizations. *IEEE Transactions on Visualization and Computer Graphics*, 29(1):504–514, 2022. doi: 10.1109/TVCG.2022.3209451 1
- [20] S. Lu. d3-annotation. <https://d3-annotation.susielu.com/>. Accessed: 2024-01-06, 01:00 UTC. 4
- [21] A. M. McNutt. No grammar to rule them all: A survey of json-style dsls for visualization. *IEEE Transactions on Visualization and Computer Graphics*, 29(1):160–170, 2022. doi: 10.1109/TVCG.2022.3209460 1
- [22] T. Munzner. *Visualization analysis and design*. CRC press, Boca Raton FL, 2014. doi: 10.1201/b17511 1
- [23] P. Parsons. Understanding data visualization design practice. *IEEE Transactions on Visualization and Computer Graphics*, 28(1):665–675, 2021. doi: 10.1109/TVCG.2021.3114959 1
- [24] A. J. Perlis. Special feature: Epigrams on programming. *ACM Sigplan Notices*, 17(9):7–13, 1982. doi: 10.1145/947955.1083808 1
- [25] X. Pu and M. Kay. A probabilistic grammar of graphics. In *SIGCHI Conference on Human Factors in Computing Systems*, pp. 1–13, 2020. doi: 10.1145/3313831.3376466 1
- [26] M. D. Rahman, B. Doppalapudi, G. J. Quadri, and P. Rosen. A survey on annotations in information visualization: Empirical insights, applications, and challenges. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2410.05579 1, 2
- [27] M. D. Rahman, G. J. Quadri, B. Doppalapudi, D. A. Szafr, and P. Rosen. A qualitative analysis of common practices in annotations: A taxonomy and design space. *IEEE Transactions on Visualization and Computer Graphics*, 2024. doi: 10.1109/TVCG.2024.3456359 1, 2, 4
- [28] M. D. Rahman, G. J. Quadri, and P. Rosen. A qualitative evaluation and taxonomy of student annotations on bar charts. *VisComm Workshop at IEEE VIS*, 2022. 2
- [29] M. D. Rahman, G. J. Quadri, and P. Rosen. Exploring annotation strategies in professional visualizations: Insights from prominent us news portals. *VisComm Workshop at IEEE VIS*, 2023. doi: 10.31219/osf.io/fd8zj 2
- [30] M. D. Rahman, G. J. Quadri, D. A. Szafr, and P. Rosen. Exploring annotation taxonomy in grouped bar charts: A qualitative classroom study. *Information Visualization*, p. 14738716241270247, 2024. doi: 10.1177/14738716241270247 1
- [31] D. Ren, M. Brehmer, B. Lee, T. Höllerer, and E. K. Choe. Chartaccent: Annotation for data-driven storytelling. In *IEEE Pacific Visualization Symposium*, pp. 230–239, 2017. doi: 10.1109/PACIFICVIS.2017.8031599 2
- [32] A. Satyanarayan, D. Moritz, K. Wongsuphasawat, and J. Heer. Vega-lite: A grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):341–350, 2016. doi: 10.1109/TVCG.2016.2599030 1, 4
- [33] A. Satyanarayan, R. Russell, J. Hoffswell, and J. Heer. Reactive vega: A streaming dataflow architecture for declarative interactive visualization. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):659–668, 2015. doi: 10.1109/TVCG.2015.2467091 4
- [34] Y. B. Shrinivasan, D. Gotzy, and J. Lu. Connecting the dots in visual analysis. In *IEEE Conference on Visual Analytics Science and Technology*, pp. 123–130, 2009. doi: 10.1109/VAST.2009.5333023 1
- [35] R. Sicat, J. Li, J. Choi, M. Cordeil, W.-K. Jeong, B. Bach, and H. Pfister. Dxr: A toolkit for building immersive data visualizations. *IEEE Transactions on Visualization and Computer Graphics*, 25(1):715–725, 2018. doi: 10.1109/TVCG.2018.2865152 2
- [36] S. Tan. Four charts that analyze how omicron’s wave compares to previous coronavirus peaks, January 2022. The Washington Post. 1
- [37] Vega. Vega-label. 3
- [38] H. Wickham. ggplot2. *Wiley Interdisciplinary Reviews: Computational Statistics*, 3(2):180–185, 2011. doi: 10.1007/978-0-387-98141-3 1, 4
- [39] L. Wilkinson. The grammar of graphics. In *Handbook of computational statistics: Concepts and methods*, pp. 375–414. Springer, 2011. doi: 10.1007/978-3-642-21551-3_13 1, 2
- [40] W. Willett, J. Heer, J. Hellerstein, and M. Agrawala. Commentspace: structured support for collaborative visual analysis. In *SIGCHI Conference on Human Factors in Computing Systems*, pp. 3131–3140, 2011. doi: 10.1145/1978942.1979407 2
- [41] J. Zhao, M. Glueck, S. Breslav, F. Chevalier, and A. Khan. Annotation graphs: A graph-based visualization for meta-analysis of data based on user-authored annotations. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):261–270, 2016. doi: 10.1109/TVCG.2016.2598543 2
- [42] J. Zong, J. Pollock, D. Wootton, and A. Satyanarayan. Animated vega-lite: Unifying animation with a grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 29(1):149–159, 2022. doi: 10.1109/TVCG.2022.3209369 1